Verified Admission Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development to implement this plan task-by-task. Steps use checkbox (`- [ ]`) syntax for tracking.

Goal: Add the owner-approved Ready with verification handshake to the existing Task 6 admission gate and Task 7 candidate transport without activating automation.

Architecture: Authenticate a deliberate request for fixed expected commits, serialize its durable operation, verify an observed native Draft→Ready window, and persist admission before an explicit single-use CI continuation. Extend the existing candidate/journal/controller contracts with versioned admission identity; retain one shared admission gate and authenticated result transport. Production remains denied until a separately selected backend, issuer, writer coverage and effective permissions are proven.

Tech Stack: Existing JavaScript ES modules, Node 24.19.0, npm 11.21.1, node:test, injected HTTP/archive/store ports, inactive GitHub Actions YAML.

Spec: Owner-approved verified admission design. Supporting runtime capability record.

Status: OWNER APPROVED - owner explicitly approved this implementation plan on 2026-10-22. This document performs no implementation, remote operation, grant, credential selection, spending or activation.

Contents

Portable approval record
Global constraints
Review focus
Baseline and acceptance precondition
File map and shared contracts
Task A: Version and authenticate admission identity
Task B: Persist single-use operations and recovery
Task C: Verify the native transition window
Task D: Compose the existing candidate gate
Task E: Continue CI explicitly and once
Task F: Inactive command wiring and acceptance
Production verification gates
Review and execution handoff

Portable approval record

Repository copy of the owner-approved 2026-10-22 amendment and implementation plan. Approval-source SHA256: design `b12db59636fe472fc494aef41a66396dc7b1bdfcbe32cd32b5a9ffff391be873`; original approved plan `892e1300716085e34341da35c5ea9a7a477a0945d7eb16113f2e8f88fc5bbf16`; approval-metadata plan revision `e5f98233dbb9fb92b6a2705acbd1fd2cbeaded994dacb23161afc92a1c9fa568`.

This portable revision changes approval metadata and links only. Supporting links point to repository-maintained capability records rather than local planning evidence. Requirements remain the approved amendment; historical approval and prerequisite descriptions are not claims of live activation or hosted acceptance. Offline implementation is staged on a Draft branch, pending exact-source review and delivery. Final exports follow the project PDF convention.

Global constraints

- The candidate is bound at verified admission, not reconstructed as an event-time snapshot.
- Ambiguity or any detected intervening candidate/readiness change denies admission.
- All worker starts and consequential completions revalidate these facts.
- Unknown outcomes remain pending; no automatic restoration to Ready is permitted.
- The native Ready ID remains part of the candidate identity; a workflow run ID is never its substitute.
- Preserve initiating human identity separately from the automation actor. `glyad` remains the sole merger.
- A native Ready button click alone never admits CI. Recovery requires human reconciliation and a deliberate return to Draft, followed by a new request when the previous obligation has been resolved.
- Before/after reads are not atomic or linearizable evidence. Unobserved `B/H → B2/H2 → B/H` can evade observation. The protocol never attests uninterrupted equality, event-time SHAs, unique mutation causality, or readiness CAS.
- No production backend, signing key, PAT/App identity, credential, positive review limit or funding is selected. No paid request is needed for this plan's offline implementation.
- Preserve Actions disabled, executable false gates, setup closed, ordinary Draft-only work and integration false. A later live stage requires separate authorization and evidence.
- Preserve Task 5 checks and package additions when composing changes. Do not merge a sibling task branch as parent synchronization.
- No rewrite of the full eleven-task setup plan, product implementation, review proxy, paid settlement or runner isolation in this increment. Their existing Task 7 obligations remain open.

Review focus

Each initially uncovered condition gets an explicit test below; these are review priorities, not exemptions from testing.

1. An authenticated rerun changes dispatch inputs or triggering actor: retain the original request and reject substitutions (A/B).
2. A store call returns `uncertain` without throwing, then the process dies: retain exclusion/pending obligation across restart (B).
3. A shared bot produces a plausible Ready witness outside the registered controller path: actor-name equality grants no authority (C/F).
4. An old journal/result payload parses successfully after the schema upgrade: no legacy record acquires new-protocol admission (A/D).
5. Explicit CI dispatch succeeds but its response is lost and no bot Ready event arrives: reconcile the original claim, never dispatch twice (E).

Baseline and acceptance precondition

Read the actual checkout's `AGENTS.md`, `docs/project-instructions.md`, `docs/project-model-policy.md`, approved `docs/superpowers/specs/2026-10-20-repository-and-project-management-design.md`, and Tasks 6-7 in `docs/superpowers/plans/2026-10-20-repository-and-project-management-setup.md`. Actual checkout at authoring: `/Users/davidkossoglyad/Documents/GitHub/glyad/github-workspace-manager`; no writes there are authorized by this planning pass.

Executable interface reference: `work/repository-setup-runtime-bridge-staging` in this mirror. Runtime bridge contract, Task 6 live capability gates, and Task 7 worker activation gates remain dependencies, not completion evidence.

Before starting A, obtain the current bridge's accepted source manifest and completed Sol/High self-review plus fresh Astra/High independent recheck. The ongoing bridge review has identified uncertain-return lock-outage and internal Draft-GET drift defects. The parent reports the bridge fix Sol/High self-review passed; fresh Astra/High independent recheck remains pending. Their fixes and acceptance belong to that existing review; this plan neither re-audits nor fixes them. Do not build on its earlier passing-test report as if those findings were closed. If acceptance is missing, retain this prerequisite as blocked while reviewing the plan.

Execution, if later approved, uses an isolated branch/worktree from the then-current accepted parent. Reconcile the accepted bridge into that baseline explicitly; record exact base/head and source hashes. Follow required literal parent synchronization, signed commits, local checks and immediately pushed synchronization within the separately authorized delivery scope. The commit checkpoints below are future checkpoints, not current permission to commit or push.

File map and shared contracts

All implementation paths below are relative to the future execution checkout. New files are proposals; existing files were inspected in staging.

Files	Responsibility
New `tools/governance/readiness-request.mjs`	Bounded dispatch authentication and canonical request identity
New `tools/governance/readiness-operation.mjs`	Durable intent, mutation state, acceptance and recovery rules
New `tools/governance/readiness-handshake.mjs`	Native Draft/Ready observations and witness verification
Modify `journal.mjs`, `controller.mjs`, `admission.mjs`, `contracts.mjs` under `tools/governance/`	Versioned identity and existing shared admission decisions
Modify `github-port.mjs`, `controller-runtime.mjs`, `worker-transport.mjs`, `worker-results.mjs`, `review-budget.mjs` under `tools/governance/`	Authenticated observation/result integration; preserve budget scope
New `tools/governance/worker-dispatch.mjs`	Single-use CI continuation claim and reconciliation only
Modify `.github/workflows/governance-controller.yml`, `.github/workflows/candidate-ci.yml`	Inactive dispatch inputs/continuation boundary
New focused tests named in A-F; update existing journal/controller/workers/runtime tests	Synthetic transport, process recovery and regression evidence

Files	Responsibility
Modify `docs/governance/{runtime-bridge,workflow-runbook,activation-runbook}.md`, `package.json`	Command semantics, capability evidence and additive format coverage

Use existing `Result<T>` with `verified`, `blocked`, `conflict`, `uncertain`; no untrusted input can supply a verified wrapper. Names below are the new interface contract, not claims that code already exists.

- `ReadinessRequest = {protocolVersion:2, requestId, repositoryId, pr, prNodeId, baseRef, base, head, initiatingActorId, originalRunId, workflowId, definitionSha}.pr` is a positive safe integer; SHAs are lowercase 40-hex; request ID is 1-128 ASCII alphanumeric/underscore/hyphen; other IDs are nonempty bounded strings (128 bytes); base ref is a validated `refs/heads/…` ref (255 bytes maximum). Reject missing/extra fields and oversized input. Expected pair/ref never comes from a fresh PR read substituted for user input.
- `VerifiedCandidateV2 = {repositoryId, pr, base, head, attempt, baseRef, requestId, protocolVersion:2}`. Preserve existing `attempt` as native Ready event ID. Identity compares every field. `prNodeId` stays in authenticated request evidence and must match the freshly resolved PR number/repository. Existing v1 candidates remain parseable for historical reconciliation only, never newly admissible under v2.
- `RunIdentity` retains the existing port's `id`, `attempt`, `repositoryId`, `actorId`, `workflowId`, `sha` (definition SHA), `executionSha`, `ref`, `event`. Verify original run separately from rerun attempt and initiating actor separately from executing/retriggering actor. Pin the approved default-branch definition; `run.head_sha` or branch name alone is insufficient.
- `ReadinessProof = {version:2, request, candidate, beforeDigest, afterDigest, mutationDigest, witnessId, originalRunId, runAttempt, definitionSha}`; exact canonical bytes are authenticated by the injected archive resolver. Digests bind complete bounded evidence; raw comments/run URLs are not signatures. Durable membership detects removed tail/history, not just changed remaining bytes.
- `Operation = {version:2, request, state, beforeDigest, mutationDigest, proofDigest, continuation}`; state is `intent | mutation-pending | mutation-observed | accepted | quarantined | reconciled`. Null digest/continuation is permitted only before its owning phase. Transitions are append-only and monotonic; `reconciled` retains previous facts and cannot erase a claim.
- Existing `admitCandidate(port,event)`, `decideReadiness(observed,event,journal)`, `reconcileTransition(port,decision)` remain the only admission path. Existing `withPrLock`, `observe`, `readJournal`, `append`, `draft`, `verifyWorker` remain composed rather than replaced by another policy engine.

Task A: Version and authenticate admission identity

Files: Create `tools/governance/readiness-request.mjs`, `tests/governance/readiness-request.test.mjs`; modify `tools/governance/{contracts,journal,worker-transport}.mjs` and `tests/governance/{journal,workers}.test.mjs`.

Interfaces: `authenticateReadinessRequest(body, context) -> Promise<Result<ReadinessRequest>>`; context supplies trusted run lookup, exact-body evidence verification, configured readiness-authorized actor IDs and immutable approved definition. `validCandidate`, `sameCandidate`, `operationKey`, journal and worker envelope parsing consume the versioned union above. V2 operation keys hash canonical ordered full identity plus action; v1 retains its historical key algorithm.

- [ ] Write tests `reject_untrusted_ref_or_definition`, `reject_changed_rerun_inputs`, `reject_unauthorized_initiator`, `v2_identity_includes_ref_request_version`, `v1_cannot_be_promoted`: assert blocked authentication, zero writes, unequal identity/keys on each changed field and no v1→v2 implicit conversion. Add malformed/extra fields, bounds, PR-node mismatch and forged provenance assertions.
- [ ] Run `node --test tests/governance/readiness-request.test.mjs tests/governance/journal.test.mjs tests/governance/workers.test.mjs`; retain actual failing assertions against absent v2 support, not only import errors.
- [ ] Implement the authentication function and version-discriminated schemas. Extend journal envelope to version 2 for v2 candidates. Permit `workflow_dispatch` journal provenance only when authenticated request/run/definition bindings are satisfied; do not simply add it to the event allowlist. Return original request identity on valid reruns, with attempt attribution separately verified.
- [ ] Run the same focused command; require all pass, including unchanged v1 historical-parser tests. Update exact-byte envelope tests for v2.
- [ ] Run `npm run check:local`, complete assigned reviews, then record the scoped signed commit `feat(governance): version verified admission identity` when delivery is authorized.

Task B: Persist single-use operations and recovery

Files: Create `tools/governance/readiness-operation.mjs`, `tests/governance/readiness-operation.test.mjs`; extend the accepted `tools/governance/local-durable-store.mjs` only for the new operation records/test backend contract, not unresolved bridge fixes.

Interfaces: `beginReadinessOperation(request, store) -> Promise<Result<Operation>>`, `recordReadinessPhase(requestId, phase, evidence, store) -> Promise<Result<Operation>>`, `reconcileReadinessOperation(requestId, authenticatedEvidence, store) -> Promise<Result<Operation>>`. Store supplies `withPrLock(key,fn)`, `readOperation(requestId)`, `appendOperation(record)`, `readObligations(key)`; reads/writes return authenticated `Result`s. Calls run within one shared PR serialization domain, never nested acquisition. An existing operation with identical request may be inspected/reconciled; it never grants a fresh mutation permit.

- [ ] Write tests `duplicate_request_has_one_mutation_permit`, `uncertain_return_retains_obligation_after_restart`, `crash_before_send_is_not_blindly_retried`, `deleted_tail_denies_recovery`, `rerun_never_refunds_or_replaces_request`. Assert one persisted intent; pending/quarantine survives process restart and explicit uncertain returns; competing process cannot pass; missing evidence never yields accepted state.
- [ ] Run `node --test tests/governance/readiness-operation.test.mjs`; capture failing behavioral assertions.
- [ ] Implement monotonic records. Persist and verify intent plus mutation-pending before sending. Persist unresolved obligations before ownership release, or retain exclusion if persistence fails. An uncertain transport result stays pending until authenticated evidence resolves that original outcome; no timeout reclaim, generic unlock, second Ready call or fresh budget. Recovery cannot promote an unsettled mutation merely because Ready is currently visible.
- [ ] Run the focused command with actual disposable local subprocess kill/restart and injected read/write outages; require pass. Label local filesystem evidence local-only; test API conformance through an injected store without pretending two runner directories share storage.
- [ ] Run local checks and assigned reviews, then scoped signed commit `feat(governance): persist readiness operation recovery` within authorized delivery.

Task C: Verify the native transition window

Files: Create `tools/governance/readiness-handshake.mjs`, `tests/governance/readiness-handshake.test.mjs`; modify `tools/governance/github-port.mjs`.

Interfaces: `verifyReadinessWindow(request, before, mutation, after, trust) -> Result<ReadinessProof>`; `runReadinessHandshake(request, port) -> Promise<Result<ReadinessProof>>`. Add port `observeReadinessWindow(request) -> Promise<Result<Window>>`, `markReady(request) -> Promise<Result<MutationEvidence>>`, `publishReadinessProof(proof) -> Promise<Result<ReadinessProof>>`. `Window` contains PR identity/state/draft/baseRef/base/head, ordered native lifecycle events, completeness and evidence digest. Existing bounded/paginated transport remains injected; unsupported completeness blocks.

- [ ] Write tests `stale_pair_while_queued`, `base_only_or_ref_drift`, `manual_ready_before_intent`, `multiple_ready_or_lifecycle_boundaries`, `missing_or_unstable_pages`, `wrong_or_uncovered_actor`, `lost_mutation_response`, `proof_write_readback_failure`, `aba_does_not_claim_event_time_equality`. Assert no retargeting, no admission on ambiguity, exactly one mutation maximum, and truthfully limited evidence fields even when an unobserved ABA fixture passes the admission-time observations.
- [ ] Run `node --test tests/governance/readiness-handshake.test.mjs`; retain failing behavior output.
- [ ] Implement Draft snapshot and full boundary persistence via B, then immediately re-read the exact precondition before one `markPullRequestReadyForReview` call. Use `prNodeId` and request ID as `clientMutationId`; an echoed ID is not idempotency or native witness evidence. On settled success require open Ready, identical pair/ref, exactly one new Ready event, expected covered controller actor, no detected intervening candidate/Draft/close/reopen change, and latest-event recheck. Never sort solely by delivery/timestamp. Authenticate exact proof bytes and readback before `accepted`.
- [ ] Run focused tests; verify a successful fixture binds all request/observation/mutation/witness/run fields and duplicate/recovery paths send no second mutation. A shared actor with unknown writer coverage remains blocked; claiming unique cause from actor/time is forbidden.
- [ ] Run local checks and assigned reviews, then scoped signed commit `feat(governance): verify readiness transition window` within authorized delivery.

Task D: Compose the existing candidate gate

Files: Modify `tools/governance/{admission,controller,controller-runtime,github-port,worker-results,worker-transport,review-budget}.mjs`, `tests/governance/{controller,runtime-bridge,workers,review-budget}.test.mjs`; create `tests/governance/verified-admission.test.mjs`.

Interfaces: `observed.admission = {proof, candidate, verified:true}` is produced only after authenticated proof and current native revalidation; never copy this bit from input. Add internal event `verified_readiness` carrying authenticated proof reference. `handleTrustedReadinessRequest({body,archive}, context) -> Promise<Result<VerifiedCandidateV2>>` authenticates A, coordinates B/C and invokes the existing gate. Refactor a lock-held internal gate helper so the handshake and acceptance share ownership without recursively acquiring `withPrLock`; public worker/controller calls retain that same lock namespace. For v2 CI, `verifyWorkerCallback(event, expectedRun)` and `verifyWorker(event)` in `github-port.mjs` must resolve their authoritative expected run from the authenticated durable continuation record introduced in E, rather than the caller-provided `workers.ci` mapping. Task D owns this callback/gate integration; Task E owns its store/HTTP providers. Match claim ID, request ID, full candidate, trusted definition SHA, run ID and exact run attempt at both start and completion. A v2 `start-ci` decision is eligibility only: under the same held PR lock, call E's `consumeCiStart(claimId, run)` and return executable start permission only for its newly persisted/read-back consumption. Duplicate eligibility or an existing consumption record grants no fresh execution. Missing or unbound continuation denies v2 CI callbacks; unrelated legacy review attribution remains separately gated.

- [ ] Write tests `native_ready_alone_never_admits`, `accepted_proof_enters_shared_gate`, `changed_pair_or_witness_denies_start`, `stale_completion_never_drafts_new_attempt`, `closure_reopen_invalidates`, `legacy_event_time_proof_not_v2`, `request_change_cannot_reset_same_ready_budget`. Assert zero admission without proof; exact latest witness/pair required at dispatch/start/completion; CI success before review; no automatic Ready call during failure handling.
- [ ] Run `node --test tests/governance/verified-admission.test.mjs tests/governance/controller.test.mjs tests/governance/workers.test.mjs tests/governance/review-budget.test.mjs`; capture failure before changing the policy.
- [ ] Replace event-time-origin admission wording/logic with authenticated admission-time origin for v2 only. Keep detected head/base/lifecycle invalidation and Draft safety pre/post revalidation/quarantine. Native metadata events can invalidate but cannot create v2 admission. Bind worker start/completion result envelopes to full v2 identity and authenticated original execution; a changed request/ref/version is stale. Retain budget counting across all base/head/request changes within the same repository/PR/native Ready attempt; readiness request ID is not a free paid-call slot.
- [ ] Run focused tests plus `node --test tests/governance/runtime-bridge.test.mjs tests/governance/journal.test.mjs`; require old safety regressions and new v2 assertions pass. Do not weaken historical tests into fictitious GitHub guarantees; update only the explicitly superseded admission semantics.
- [ ] Run local checks and assigned reviews, then scoped signed commit `feat(governance): gate workers on verified admission` within authorized delivery.

Task E: Continue CI explicitly and once

Files: Create `tools/governance/worker-dispatch.mjs`, `tests/governance/worker-dispatch.test.mjs`; modify `tools/governance/{controller-runtime,worker-transport,github-port,local-durable-store}.mjs` and `tests/governance/runtime-bridge.test.mjs`. `local-durable-store.mjs` owns local-test durable continuation/start records; `github-port.mjs` owns the injected store adapters and HTTP dispatch/discovery/verification adapters. No production backend is added.

Interfaces: `dispatchAdmittedCi(candidate, port) -> Promise<Result<DispatchClaim>>`, `reconcileCiDispatch(claimId, port) -> Promise<Result<DispatchClaim>>`. Claim is `{id,candidate,worker:'ci',requestId,definitionSha,status:'pending'|'observed'|'quarantined',run:null|RunIdentity,start:null|StartConsumption}`. `StartConsumption = {claimId,runId,runAttempt,candidate,definitionSha,state:'consumed'}` is immutable execution authority history, not a reusable permit. `github-port.mjs` provides `claimCiContinuation(candidate)`, `readCiContinuation(claimId)`, `bindCiRun(claimId, run)`, `consumeCiStart(claimId, run)` returning `Promise<Result<...>>` through `store.readCiClaim`, `store.appendCiClaim`, and `store.appendCiStart` under the existing held PR lock. The accepted local test store implements those exact methods with append/readback, durable pending obligations and authenticated local-test evidence. `dispatchCi(claim)` and `findCiRun(claim)` are concrete injected HTTP/archive adapters in `github-port.mjs`: dispatch the trusted approved definition with claim/request/full candidate inputs, then authenticate exact-body run provenance through the existing resolver. Bind the original run/attempt once with verified readback; reject conflicting bindings. API run lists alone do not bind dispatch inputs, so absent issuer evidence blocks discovery rather than selecting a nearby run. The callback gate in D loads this same durable binding. `consumeCiStart` grants a new permission only on the first verified durable consumption; subsequent calls deny even for the identical run. Persist-before-return uncertainty consumes or quarantines the slot and never permits execution on retry. Run discovery must authenticate request/claim/full candidate/definition binding; a nearest timestamp or same head is insufficient.

- [ ] Write tests `bot_event_suppressed_explicit_dispatch_still_works`, `duplicate_continuation_sends_once`, `dispatch_success_lost_response_reconciles`, `unknown_run_keeps_claim_occupied`, `drift_before_dispatch_denies`, `forged_run_cannot_claim_completion`. Assert one send maximum, durable pending before send, zero blind retry, stale candidate denied, and zero review/API requests. Add composed-adapter tests `composed_dispatch_binds_callback_gate`, `composed_lost_response_discovers_original_run`, `ambiguous_or_unavailable_discovery_retains_claim`, `run_or_attempt_substitution_denied_at_start_and_completion`, `concurrent_duplicate_starts_grant_one_permission`, and `ci_workflow_rerun_cannot_replace_claimed_run_or_restart`. Instantiate `createGithubPort` with the actual local durable test store and synthetic HTTP/archive inputs; do not mock its continuation methods. Assert the persisted binding reaches D's existing callback gate, exactly one executable permission is returned under concurrency, a worker rerun with the same run ID but a different attempt is denied, and reconciliation never returns another start permission.
- [ ] Run `node --test tests/governance/worker-dispatch.test.mjs`; capture failing assertions.
- [ ] Implement claim under the same PR domain after proof acceptance and fresh admission. Dispatch through injected trusted explicit workflow dispatch/connected continuation; never rely on a bot-generated Ready event. Record/read back the exact run binding through the concrete providers above, and consume a start only after D's fresh shared admission succeeds. Both start and completion authenticate the original bound run/attempt; completion additionally requires its persisted start-consumption record. An original execution can finish or be reconciled after uncertainty without replacing that identity or renewing execution authority. Interrupted send stays occupied until reconciled. CI continuation does not create a paid reservation; future review uses existing Task 7 reservation/claim gates, acquiring funding-scope lock before PR lock consistently wherever both are needed.
- [ ] Run focused tests, including process restart between claim and send, after send/before response, and after durable start consumption/before permission delivery; require pass. Run `node --test tests/governance/runtime-bridge.test.mjs` to verify the real offline port composition. Repeated worker-start callbacks and workflow reruns must remain denied after restart, including when the first start response was lost. Reconciliation observes an already-created run without granting another execution.
- [ ] Run local checks and assigned reviews, then scoped signed commit `feat(governance): continue admitted CI once` within authorized delivery.

Task F: Inactive command wiring and acceptance

Files: Modify `.github/workflows/{governance-controller,candidate-ci}.yml`, `docs/governance/{workflow-runbook,activation-runbook,runtime-bridge}.md`, `package.json`; create `tests/governance/readiness-command.test.mjs`.

Interfaces: Inactive `workflow_dispatch` inputs are `pr`, `expected_base_ref`, `expected_base_sha`, `expected_head_sha`, `request_id`, `protocol_version` (exact value `2`). Repository/actor/node/run/definition identities come from verified trusted context, never user-supplied authority. Runtime calls D then E only after all gates pass. Task F owns dormant wiring of `createGithubPort` to the injected store/HTTP/archive providers from E and routes the dispatched CI start/completion through D's `handleTrustedWorkerCallback`, using claim ID to resolve the durable expected run. Worker-supplied run/candidate fields are assertions to verify, never the configured authority. No CI execution step is reachable without a newly consumed start permission; retries/reruns reconcile or report denial. Command messages are “Ready and admitted for these commits” or “Not admitted; reconciliation required.” No command wrapper is necessary for the first increment.

- [ ] Write tests `inactive_workflow_cannot_mutate_or_dispatch`, `input_cannot_choose_new_candidate`, `branch_definition_rejected`, `complete_offline_handshake_continuation`, `native_button_explains_blocked_state`. Assert false gates and default production denial persist, request strings are data rather than shell interpolation, no PR checkout/config execution occurs in the privileged controller, and the positive synthetic path reaches exactly one authenticated CI start. Exercise this path with the same concrete `createGithubPort`/local-test-store composition intended by the dormant wiring, synthetic HTTP/archive responses and no mocked continuation methods. Assert lost-response discovery binds the original callback identity, ambiguous discovery stays pending, substituted run/attempt is denied at both callback boundaries, and duplicate starts or a CI workflow rerun cannot reach execution again.
- [ ] Run `node --test tests/governance/readiness-command.test.mjs`; capture expected failure on missing inactive input/wiring contract.
- [ ] Add dormant dispatch contract and trusted invocation wiring while retaining `if: ${{ false }}` and failing production capability preflight. Document actor allowlist, shared writer inventory, original/rerun attribution, human Draft recovery, no automatic restoration and the limited proof. Add new files to existing formatting coverage without replacing other tasks' paths. Every Markdown file gets a linked ToC.
- [ ] Run `node --test tests/governance/readiness-command.test.mjs` and `npm run check:local`; require pass. Record exact manifest, full local counts and no-key/no-network evidence. A test fixture's positive path is not hosted acceptance.
- [ ] Complete Sol/High self-review and fresh Astra/High independent task review; freeze reviewed artifacts. Within separately authorized delivery, signed commit `feat(governance): stage verified readiness command`, followed by whole-branch review against exact base/head. Do not mark Ready or enable Actions as part of delivery.

Production verification gates

These are prerequisites for a later separately authorized production stage, not implementation tasks silently funded or enabled by this plan.

Gate	Required evidence before production can be considered
Durable backend selection	Owner/security selection of a supported shared backend and its scope/permissions; demonstrated cross-runner serialization, durable claims/obligations, conditional updates, crashes/outages and explicit recovery. Local fsync tests and Actions concurrency alone fail this gate. No backend is selected here.
Exact-body provenance	Supported issuer/resolver verification of manifest/journal/result bytes, immutable membership including deleted tails, original run/attempt, repository and approved definition. Existing ephemeral signed fixtures prove a protocol only, not a GitHub native signature service.

Gate	Required evidence before production can be considered
Readiness writer coverage	Inventory every workflow/identity able to write Ready as the chosen actor; prove all trusted writers obey the same domain. Actor name/time cannot identify one request uniquely. Preserve the approved admission-time limitation; if stronger unique causality becomes required, stop for redesign.
Effective permissions	Verify metadata/readiness, ledger Issues, Actions reads and explicit dispatch using the selected existing identity if supported. Any Contents writes, attestation grants, isolated App or new credential is a separate decision. No broad grant inferred from manual dispatch.
Live native evidence	Separately authorized disposable probes establish pagination/completeness behavior, mutation return/readback, native event shape and trusted definition/run bindings. Unknown capability fails closed. No production failure injection.
Remaining Task 7 work	Paid worker claim/settlement, actual resource enforcement, isolated runner/proxy/material extraction and result issuer still follow the existing gap map. This increment does not claim they are finished.
Funding and activation	Later explicit owner choices for positive resource limits, spending exposure, secure credential destination and activation. Only then hosted acceptance with authenticated `governance/ci`, `governance/review`, `governance/admission` before enforcement. Included app usage is not API funding.

Review and execution handoff

The selected method is subagent-driven: Sol/Medium implementation, Sol/High implementation self-review, fresh Astra/High independent review/fix rechecks and whole-branch review. Planning is Astra/Medium; the separate elevated Astra/High round-1 recheck completed with PASS. Approved-document PDF rendering is Sol/Medium with Sol/High rendering self-review. No fallback model is silently substituted.

Coverage check: request and identity → A; durable single-use/reconciliation → B; observation window, race truthfulness and proof → C; shared candidate gates and invalidation → D; bot-event suppression and dispatch uncertainty → E; inactive UX, default-branch trust and end-to-end fixtures → F; unselected production guarantees → gates above. All five review-focus items have named tests. This is the author's bounded consistency check. The elevated Astra/High round-1 recheck completed with PASS; independent code review remains part of execution.

The owner explicitly approved this implementation plan on 2026-10-22. Existing design approval and execution-method preference are preserved; approval of this plan is recorded separately. Do not re-ask for an execution-method choice. Report any unmet bridge acceptance or production evidence separately without reducing the approved requirements.